

Per-Quote Indexable URLs

Giving every sourced quotation its own crawlable page — the library's single biggest organic-search lever, and the one most able to backfire if built as thin pages on an unstable slug.

Against the current stack: SvelteKit 5 static-adapter (`prerender` + `entries()`) + Django/DRF. 267 reviewed quotations across 10 authors today; an ~50-per-author goal ahead.

THE ONE DECISION THAT GATES THE BUILD

The quote `slug` today is *author-slug + a hash of the text*. But your text is actively repaired — the `english-qa` and `book-import` skills fix OCR slips and punctuation. If a URL is minted from the text, **every repair silently changes the URL** and throws away that page's ranking equity and every inbound link. Freeze a stable identifier *before* a single URL is exposed. Everything else here is reversible; this is not.

THE CHANGE

What it is

A dedicated, crawlable page per quotation — `/quotes/<author>/<quote>/` — instead of quotes living only as `<li>` rows inside the author page.

The data model is already 90% ready: `Quote` has a unique slug, a `reviewed` publication gate, and a resolvable source (chapter/sermon + paragraph). The backend gains a single-quote view beside `QuotePageView`; the frontend gets a `[quote]` route with `prerender = true` + an `entries()` generator — the exact machinery the chapter pages already use. ~267 static pages today; low thousands at the 50/author goal, which a static build handles easily (books already prerender far more).

SEO UNDERSTANDING

Why it matters

1 It matches how quotes are actually searched

The dominant query is the *exact string*, pasted in. A list row is a weak target; a page with its own title, description, canonical and `Quotation` JSON-LD is a strong one — ~267 focused pages instead of ~11 list pages.

2 You own the answer nobody else sources

Unsourced aggregators rank for these strings now. A page that *is* the citation (book, chapter, paragraph, full-text link) is the E-E-A-T edge — and the natural citation target for AI answer engines, which increasingly quote with attribution.

3 It anchors every other quote win

Per-quote `og:image`, a real share destination (today "Copy" points at a book paragraph, not a quote page), and Google's quote treatments all need a canonical per-quote URL to attach to.

4 It densifies the internal-link graph

Each page links up to the author's quotes hub, out to the work, and across to the bio — turning a flat list into a cluster that lifts the whole `/quotes` neighbourhood.

RECOMMENDED — A STABLE, TEXT-INDEPENDENT SLUG

Mint the slug **once at creation and store it**; never recompute it from text. Freeze the current `author-slug + hash` value into a column and stop re-deriving it (readable, keyword-bearing), rather than switching to an opaque id. The non-negotiable: a text edit must not move the URL. Add a `301` redirect map for any slug that has already shifted, and treat an unpublish (`reviewed` → false) as a redirect / `410` to the author page — not a bare `404`.

THIN-CONTENT MITIGATION

The page is a passage, not a one-liner

A page whose body is one sentence is the textbook thin page — Google crawls it and files it under “Crawled, currently not indexed.” You avoid that because you already resolve the source paragraph (the `?p=` unit). Each page carries:

- ✓ The quote as the hero, with author and the **exact citation** (chapter ¶).
- ✓ The **surrounding paragraph** as context (public-domain source), with “Read the full chapter →”.
- ✓ A short **author card** — portrait, one line, link to the bio (reuses the Person entity from the last PR).
- ✓ **3–6 related quotations** by the same author (internal links).
- ✓ Share affordances — copy-with-attribution (already built) and a quote-card image.

That converts a one-liner into a genuine, linkable passage page — the difference between indexed and ignored.

CANONICALS — ONE JOB PER SURFACE

URL	TARGETS	CANONICAL
<code>/quotes/<a>/<q>/</code>	the quote (exact string)	self
<code>/quotes/<a>/</code>	the collection	self
<code>/books/<s>/<n>/</code>	the reading	self

The same sentence now lives on three of your URLs. Self-canonical each, and frame the author page as a *collection* (the `ItemList` shipped last PR) so it never competes with the individual pages for the exact string.

RISKS & MITIGATIONS

RISK	MITIGATION
Unstable slug	GATING Freeze slug at creation; 301 map (p.1).
Thin content	Passage layout: paragraph + related + bio.
Self-competition	Self-canonicals; author page = collection.
Unpublish 404s	Redirect / 410 to author page, not bare 404.
Near-twin pages	Unique context paragraph + related set differ.
Sitemap bloat	Static prerender scales; watch past ~10k.

SEQUENCE

How to build it — five waves

0 Decide & freeze the slug **GATE — NOTHING SHIPS BEFORE THIS**

Add the stable-slug column + backfill; write the redirect map. This is the only step expensive to undo.

1 Backend

Single-quote endpoint beside `QuotePageView` — quote + resolved source paragraph + neighbouring quotes + author blurb. Keep the `reviewed` gate; return 404 / 410 correctly.

2 Frontend route

`/quotes/[author]/[quote]/` with `prerender` + `entries()` from the reviewed set; passage layout; `Seo.svelte` head (self-canonical, `Quotation` + author `Person @id`, reusing this PR’s helpers — `workHref` is already extracted); per-quote `og:image`.

3 Wire the graph

Link each list-page quote to its page; add a “Quotes from this book” block on book pages; add every quote URL to `sitemap.ts` (English-only, matching the current quote pages).

4 Validate before scaling

Ship **one author** first; confirm indexation in Search Console (Crawled vs Indexed) and rich-result validity, then generate the rest. Gates as ever: `check` + `test` green, `verify-local`, `/simplify` + `/code-review high`, squash-merge on green.

If you do one thing first: freeze the slug. It is the only step that is expensive to undo — everything after it can be iterated in the open.